

NOTES

Tuples, Set, Dictionaries & Its Function



Tuples, Set, Dictionaries & Its Methods

1. Tuples

1.1 Introduction

A tuple in Python is an immutable, ordered collection of items. Tuples can hold a variety of data types, including integers, floats, strings, and even other tuples or lists.

Key Characteristics of Tuples:

- **Immutable:** Once a tuple is created, its elements cannot be changed, added, or removed. This makes tuples suitable for storing data that should not be modified.
- **Ordered:** The items in a tuple have a defined order, meaning that you can access them using indexing (similar to lists).
- **Heterogeneous:** A tuple can contain elements of different data types.
- **Defined by Parentheses:** Tuples are created by placing comma-separated values within parentheses `()`.

1.2 Creating a Tuple:

You can create a tuple using parentheses, with elements separated by commas:

1.2.1 Declaration of Tuple

```
Python
# A tuple with integers
my_tuple = (1, 2, 3)

# A tuple with mixed data types
mixed_tuple = (1, "apple", 3.14, True)

# A nested tuple
nested_tuple = (1, (2, 3), (4, 5))

# An empty tuple
empty_tuple = ()

# A tuple with a single element (note the trailing comma)
single_element_tuple = (42,)
```

1.2.2 Accessing Tuple Elements:

You can access elements of a tuple using indexing, just like lists.

```
Python
my_tuple = (1, 2, 3, "apple")

print(my_tuple[0]) # Output: 1
print(my_tuple[-1]) # Output: 'apple' (negative indexing)
```

1.3 Tuple Operations:

1.3.1 Concatenation:

You can concatenate two tuples using the `+` operator.

```
Python
tuple1 = (1, 2)
tuple2 = (3, 4)
combined = tuple1 + tuple2
print(combined) # Output: (1, 2, 3, 4)
```

1.3.2 Repetition:

You can repeat a tuple using the `\*` operator.

```
Python
repeated = tuple1 * 3
print(repeated) # Output: (1, 2, 1, 2, 1, 2)
```

1.3.3 Membership:

Use the `in` keyword to check for membership.

```
Python
print(3 in combined) # Output: True
```

1.3.4 Tuple Length:

You can find the number of elements in a tuple using the `len()` function.

```
Python
print(len(my_tuple)) # Output: 4 (for the tuple (1, 2, 3, "apple"))
```

2. Set

2.1 Introduction

A set in Python is an unordered collection of unique items. Sets are mutable, meaning you can add or remove items after a set is created. They are commonly used to eliminate duplicate entries and to perform mathematical set operations like union, intersection, and difference.

2.2 Key Characteristics of Sets

- **Unordered:** The elements in a set do not have a defined order. You cannot access items by their index, as you would with lists or tuples.
- **Unique Elements:** Sets automatically discard duplicate items. Each element in a set must be unique.
- **Mutable:** You can add or remove elements from a set after it has been created.
- **Defined by Curly Braces:** Sets are created using curly braces `{}` or the `set()` constructor.

2.3 Creating a Set

You can create a set using curly braces or the `set()` function.

```
Python
# Using curly braces

my_set = {1, 2, 3, 4}
```

2.3.1 Declaration Using Set Constructor

```
Python
# Using the set() constructor

another_set = set([1, 2, 3, 4])
```

2.3.2 Declaration of set with mixed data types

```
Python
# A set with mixed data types

mixed_set = {1, "apple", 3.14, True}
```

2.3.3 Declaration of set with duplicates (duplicates will be removed)

```
Python
duplicate_set = {1, 2, 2, 3} # Output: {1, 2, 3}
```

2.4 Set Operations

Sets support several operations, including:

2.4.1 Adding Elements

You can add a single element using `add()`.

```
Python
my_set.add(5)
print(my_set) # Output: {1, 2, 3, 4, 5}
```

2.4.2 Removing Elements

We can remove elements using `remove()` or `discard()`. The `remove()` method raises an error if the element is not found, while `discard()` does not.

```
Python
my_set.remove(3) # Raises an error if 3 is not in the set
my_set.discard(4) # No error if 4 is not found
```

2.4.3 Union

Combine two sets using the `union()` method or the `|` operator.

```
Python
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1.union(set2) # Output: {1, 2, 3, 4, 5}
```

2.4.4 Intersection

Get the common elements using the `intersection()` method or the `&` operator.

```
Python
intersection_set = set1.intersection(set2) # Output: {3}
```

2.4.5 Difference

Find elements in one set that are not in another using the `difference()` method or the `-` operator.

```
Python
difference_set = set1.difference(set2) # Output: {1, 2}
```

2.4.6 Symmetric Difference

Get elements that are in either set but not in both using the `symmetric_difference()` method or the `^` operator.

```
Python
sym_diff_set = set1.symmetric_difference(set2)
# Output: {1, 2, 4, 5}
```

2.5 Set Membership

You can check for membership using the `in` keyword:

Python

```
print(2 in my_set) # Output: True  
print(6 in my_set) # Output: False
```

2.6 Why and When we use Sets

- **Elimination of Duplicates:** Sets automatically discard duplicate values, making them useful for storing unique items.
- **Performance:** Sets provide fast membership testing and can be more efficient than lists for certain operations.
- **Mathematical Operations:** Sets support mathematical operations that are useful in various programming tasks.

3. Dictionaries

3.1 Introduction

A **dictionary** in Python is a built-in data structure that allows you to store data in key-value pairs. Dictionaries are mutable, meaning you can change them after their creation, and they are unordered collections, so the items do not have a defined order.

3.2 Key Characteristics of Dictionaries:

- **Key-Value Pairs:** Each item in a dictionary is a pair consisting of a **key** and a **value**. The key acts as an identifier for the associated value.
- **Unique Keys:** Keys in a dictionary must be unique; if a duplicate key is added, the previous value associated with that key will be overwritten.
- **Mutable:** You can add, remove, or change items in a dictionary after it is created.
- **Unordered:** The items in a dictionary do not maintain any order. As of Python 3.7, dictionaries preserve the insertion order, but this should not be relied upon for logic, as they are inherently unordered.
- **Defined by Curly Braces:** Dictionaries are created using curly braces `{}` or the `dict()` constructor.

3.3 Dictionaries Declarations

You can create a dictionary by placing comma-separated key-value pairs within curly braces or by using the `dict()` constructor:

3.2.1 Using curly braces

```
Python
my_dict = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}
print(my_dict)
```

3.2.2 Using the dict() constructor

```
Python
another_dict = dict(name="Bob", age=25, city="Los Angeles")
```

3.2.3 A dictionary with mixed data types

```
Python
mixed_dict = {
    1: "one",
    2: [1, 2, 3],
    3.14: (3, 14),
    "is_active": True
}
print(mixed_dict)
```

3.3 Dictionary Object Methods

Dictionaries in Python have several built-in methods that allow you to manipulate and access the data they contain. Here's a list of common dictionary methods along with a brief explanation of each:

Common Dictionary Methods

3.3.1 dict.clear()

Removes all items from the dictionary, leaving it empty.

```
Python
my_dict = {'a': 1, 'b': 2}
my_dict.clear()
print(my_dict) # Output: {}
```

3.3.2 dict.copy()

Returns a shallow copy of the dictionary.

```
Python
my_dict = {'a': 1, 'b': 2}
new_dict = my_dict.copy()
print(new_dict) # Output: {'a': 1, 'b': 2}
```

3.3.3 dict.fromkeys(seq, value=None)

Creates a new dictionary with keys from the given sequence (`seq`) and assigns them a value (default is `None`).

```
Python
keys = ('a', 'b', 'c')
new_dict = dict.fromkeys(keys, 0)
print(new_dict) # Output: {'a': 0, 'b': 0, 'c': 0}
```

3.3.4 dict.get(key, default=None)

Returns the value for the specified key if the key is in the dictionary; otherwise, it returns the default value (which is `None` if not specified).

```
Python
my_dict = {'a': 1, 'b': 2}
print(my_dict.get('a')) # Output: 1
print(my_dict.get('c', 'Not Found')) # Output: Not Found
```

3.3.5 dict.items()

Returns a view object that displays a list of a dictionary's key-value tuple pairs.

Python

```
my_dict = {'a': 1, 'b': 2}
print(my_dict.items()) # Output: dict_items([('a', 1), ('b', 2)])
```

3.3.6 dict.keys()

Returns a view object that displays a list of all the keys in the dictionary.

Python

```
my_dict = {'a': 1, 'b': 2}
print(my_dict.keys()) # Output: dict_keys(['a', 'b'])
```

3.3.7 dict.pop(key, default=None)

Removes the specified key and returns the corresponding value. If the key is not found, it returns the default value (if provided) or raises a `KeyError`.

Python

```
my_dict = {'a': 1, 'b': 2}
value = my_dict.pop('a')
print(value) # Output: 1
print(my_dict) # Output: {'b': 2}
```

3.3.8 dict.popitem()

Removes and returns an arbitrary key-value pair from the dictionary. Raises a `KeyError` if the dictionary is empty.

Python

```
my_dict = {'a': 1, 'b': 2}
item = my_dict.popitem()
print(item) # Output: ('b', 2) or ('a', 1) (order is arbitrary)
```

3.3.9 dict.setdefault(key, default=None)

Returns the value of the specified key. If the key does not exist, it inserts the key with a specified default value and returns that value.

```
Python
my_dict = {'a': 1}
print(my_dict.setdefault('b', 2)) # Output: 2
print(my_dict) # Output: {'a': 1, 'b': 2}
```

3.3.10 dict.update(other)

Updates the dictionary with elements from another dictionary or from an iterable of key-value pairs. Existing keys will have their values updated.

```
Python
my_dict = {'a': 1, 'b': 2}
my_dict.update({'b': 3, 'c': 4})
print(my_dict) # Output: {'a': 1, 'b': 3, 'c': 4}
```

3.3.11 dict.values()

Returns a view object that displays a list of all the values in the dictionary.

```
Python
my_dict = {'a': 1, 'b': 2}
print(my_dict.values()) # Output: dict_values([1, 2])
```

3.4 Dictionary Comprehensions

Dictionary comprehension in Python is a concise way to create dictionaries using a single line of code. It allows you to generate dictionaries from existing iterables (like lists or tuples) or to transform data while creating the dictionary. The syntax for dictionary comprehension is similar to list comprehension but uses curly braces `{}` instead of square brackets `[]`.

Syntax of Dictionary Comprehension:

```
Python
new_dict = {key_expression: value_expression for item in iterable if condition}
```

- **key_expression:** An expression that computes the key for the new dictionary.
- **value_expression:** An expression that computes the value associated with the key.
- **iterable:** An iterable (like a list or a range) to loop through.
- **condition (optional):** A filtering condition that determines whether the key-value pair should be included in the new dictionary.

3.4.1. Basic Dictionary Comprehension:

Creating a dictionary of squares for numbers from 0 to 4.

```
Python
squares = {x: x2 for x in range(5)}
print(squares)

# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

3.4.2. Using Conditional Logic:

Creating a dictionary of even numbers and their squares from 0 to 9.

```
Python
even_squares = {x: x2 for x in range(10) if x % 2 == 0}
print(even_squares)

# Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

3.4.3. Transforming Data:

Creating a dictionary from a list of tuples.

```
Python
items = [('apple', 1), ('banana', 2), ('cherry', 3)]
fruit_dict = {fruit: quantity for fruit, quantity in items}
print(fruit_dict) # Output: {'apple': 1, 'banana': 2, 'cherry': 3}
```

3.4.4. Swapping Keys and Values:

Creating a new dictionary by swapping keys and values.

```
Python
original_dict = {'a': 1, 'b': 2, 'c': 3}
swapped_dict = {value: key for key, value in original_dict.items()}
print(swapped_dict) # Output: {1: 'a', 2: 'b', 3: 'c'}
```

3.4.5. Nested Dictionary Comprehension:

Creating a dictionary of dictionaries, for example, to represent a grid.

```
Python
grid = {x: {y: x * y for y in range(3)} for x in range(3)}
print(grid)

# Output: {0: {0: 0, 1: 0, 2: 0}, 1: {0: 0, 1: 1, 2: 2}, 2: {0: 0, 1: 2, 2: 4}}
```

3.4.6 Benefits of Dictionary Comprehension:

- **Conciseness:** Reduces the amount of code needed to create a dictionary.
- **Readability:** Provides a clear and expressive way to generate dictionaries, making the intent more obvious.
- **Performance:** Generally faster than using traditional loops to create dictionaries due to internal optimizations.

3.5 Dictionary View Objects

In Python, dictionary view objects provide a dynamic view of the dictionary's entries, allowing you to see the dictionary's keys, values, or key-value pairs in a way that reflects any changes made to the dictionary. There are three main types of view objects in dictionaries: keys, values, and items.

3.5.1. Keys View

The keys view object provides a view of the dictionary's keys. It can be accessed using the `keys()` method.

Accessing Keys View:

```
Python
my_dict = {'a': 1, 'b': 2, 'c': 3}
keys_view = my_dict.keys()
print(keys_view) # Output: dict_keys(['a', 'b', 'c'])
```

Dynamic Updates: If the dictionary is modified, the keys view reflects these changes:

```
Python
my_dict['d'] = 4
print(keys_view) # Output: dict_keys(['a', 'b', 'c', 'd'])
```

3.5.2. Values View

The values view object provides a view of the values in the dictionary. It can be accessed using the `values()` method.

Accessing Values View:

```
Python
values_view = my_dict.values()
print(values_view) # Output: dict_values([1, 2, 3, 4])
```

Dynamic Updates: Similar to keys, if the dictionary changes, the values view reflects these changes:

```
Python
my_dict['b'] = 5
print(values_view) # Output: dict_values([1, 5, 3, 4])
```


3.5.3. Items View

The items view object provides a view of the key-value pairs in the dictionary as tuples. It can be accessed using the `items()` method.

Accessing Items View:

```
Python
items_view = my_dict.items()
print(items_view)
# Output: dict_items([('a', 1), ('b', 5), ('c', 3), ('d', 4)])
```

Dynamic Updates: As with the other views, the items view reflects changes in the dictionary:

```
Python
del my_dict['c']
print(items_view)

# Output: dict_items([('a', 1), ('b', 5), ('d', 4)])
```

3.5.4 Characteristics of Dictionary View Objects:

- **Dynamic:** Dictionary view objects provide a dynamic view of the dictionary. This means that any modifications to the dictionary (adding, removing, or updating items) are reflected in the view object without needing to recreate it.
- **Iterable:** The view objects can be iterated over in loops or converted into lists if needed:

```
Python
for key in my_dict.keys():
    print(key)

# Outputs each key in the dictionary
```

- **Set-like Behaviour:** The keys and items views support some set-like operations, such as union, intersection, and difference:

```
Python
set_of_keys = {'a', 'b'}
print(set_of_keys.intersection(keys_view))

# Output: {'a', 'b'}
```

3.6 Difference between Tuple, Set & Dictionaries

	Tuple	List	Dictionary	Set
Example	('Book 1', 12.99)	['apple', 'banana', 'orange']	{'name': 'Joe', 'age': 10}	{10, 20, 12}
Mutable?	Immutable	Mutable	Mutable	Mutable
Ordered?	Ordered	Ordered	Preserves order since Python 3.7	Unordered
Iterable?	Yes (takes linear time)	Yes (takes linear time)	Yes (constant time)	Yes (constant time)
Use case	Immutable data	Data that needs to change	Key/Value pairs	Unique items

THANK YOU

